Lecture Notes 2026

Computational Physics

Year 3 · Semester B

77315 · HUJI Physics · Eran Rehani

Contents

1 Lecture 01 — Introduction: Roundoff Errors, Numerical Differentiation, and Interpolation

Date: 26/03/2026

Overview

Computational physics combines numerical analysis methods to solve physical problems. Historically, physics progressed through a binary of theoretical physics (creating mathematical models) and experimental physics (measuring reality). Today, the computational approach serves as a formidable third branch. By leveraging the immense processing power of computers, we can solve complex models (such as nonlinear differential equations with no analytical solutions) and perform macroscopic simulations of many-body systems that would be otherwise impossible to track.

When translating pure mathematics into computer code, we must abandon the notion of "infinite precision." Three main factors must always be considered:

1. **Roundoff Errors**: Inaccuracies due to the finite memory computers use to store numbers.
2. **Truncation Errors (Accuracy)**: Inaccuracies arising from using a finite mathematical approximation (like stopping a Taylor series early).
3. **Stability**: The tendency of an algorithm to either dampen or exponentially amplify these microscopic errors over time.

1.1 Errors, Accuracy, and Stability

1.1.1 Catastrophic Cancellation and Roundoff Errors

Computers represent real numbers using a limited number of bits. A common representation is 4 bytes (Float32, yielding ~ 7 decimal digits of precision) or 8 bytes (Float64, yielding ~ 16 decimal digits).

Because of this finite representation, every basic arithmetic operation introduces a tiny relative error. While this is usually negligible, a severe problem known as **catastrophic cancellation** occurs when subtracting two very close numbers.

Pedagogical Example: The Quadratic Formula

Consider the quadratic equation $x^2 - 1000x + 0.001 = 0$. Here, $b = 500$ and $c = 0.001$. The standard quadratic formula gives the roots as $x = b \pm \sqrt{b^2 - c}$.
If we calculate the smaller root using subtraction: $x_{\text{small}} = 500 - \sqrt{250000 - 0.001} \approx 500 - 499.999999 = 0.000001$.
If our computer only has 7 digits of precision (Float32), $\sqrt{250000 - 0.001}$ will be rounded to exactly 500.0000. The subtraction $500.0000 - 500.0000$ yields 0. We have completely lost the true answer due to catastrophic cancellation!
The elegant solution: Multiply the numerator and denominator by the conjugate $b + \sqrt{b^2 - c}$. This transforms the formula to $x_{\text{small}} = \frac{c}{b+\sqrt{b^2-c}}$. Now: $x_{\text{small}} = \frac{0.001}{500+500} = \frac{0.001}{1000} = 10^{-6}$. By avoiding the subtraction of close numbers, we maintain perfect precision.

1.1.2 Truncation Error and Stability

Truncation error is the difference between the exact mathematical formula and the approximated algorithm. For example, a Taylor series is infinite, but a computer can only sum N terms. The discarded terms constitute the truncation error.

Stability dictates whether an algorithm is usable. An algorithm is unstable if tiny roundoff or truncation errors grow exponentially as the computation proceeds.

Intuitive Analogy: The Golden Mean Recursion

Consider calculating the powers of the "Golden Mean" $\phi = \frac{\sqrt{5}-1}{2} \approx 0.618$. Mathematically, ϕ satisfies the recurrence relation: $\phi^{n+1} = \phi^{n-1} - \phi^n$. This seems like a brilliantly fast way to compute ϕ^{100} without multiplication!
However, this linear recurrence relation has two mathematical roots: $\phi_1 \approx 0.618$ and $\phi_2 = \frac{-\sqrt{5}-1}{2} \approx -1.618$. Even if we start with the exact values for $n = 1, 2$, the tiny roundoff error introduced by the computer acts as a "seed" for the second root. Because $|\phi_2| > 1$, its contribution grows exponentially. By $n = 50$, the parasitic ϕ_2 term completely dominates, and the algorithm blows up to infinity. This is the hallmark of numerical instability.

1.2 Numerical Differentiation

How do we teach a computer to calculate a derivative, $\lim_{h\to 0} \frac{f(x+h)-f(x)}{h}$? We cannot set $h = 0$, nor can we make h infinitely small due to roundoff errors.

If we have a discrete grid of points $x_n = x_0 + nh$, we can approximate derivatives using Taylor expansions:

$$f(x_0 + h) = f_0 + hf' + \frac{h^2}{2}f'' + \frac{h^3}{6}f''' + \mathcal{O}(h^4)$$
$$f(x_0 - h) = f_0 - hf' + \frac{h^2}{2}f'' - \frac{h^3}{6}f''' + \mathcal{O}(h^4)$$

Subtracting the second from the first cancels the f'' term, yielding the **Central Difference Formula**:

$$f' = \frac{f_{+1} - f_{-1}}{2h} + \mathcal{O}(h^2) \tag{1}$$

Geometrically, instead of drawing a secant line from x to $x+h$ (which is biased), we draw a secant line from $x - h$ to $x + h$, which perfectly balances the slope around x.

The Step-Size Dilemma. If we make h large, the Taylor approximation fails (truncation error $e_t \sim h^2 f'''$). If we make h microscopic, catastrophic cancellation destroys the numerator (roundoff error $e_r \sim \frac{\varepsilon_f f}{h}$). Minimising $E(h) = \frac{\varepsilon_f f}{h} + h^2 f'''$ yields:

$$h_{\text{opt}} \sim \sqrt[3]{\frac{3\,\varepsilon_f\, f}{f'''}} \tag{2}$$

1.3 Interpolation

Interpolation is the art of reading "between the lines" of a discrete dataset.

- **Lagrange Interpolation**: Fits a single global polynomial of degree $N-1$ through exactly N points. High-degree polynomials suffer from **Runge's Phenomenon**: they oscillate violently at the edges of the interval.
- **Cubic Splines**: Piecewise polynomials that enforce continuity of the function value, first derivative, and second derivative across each knot. The result is a stable tridiagonal system.

The Tridiagonal System for Cubic Splines

Let $h_i = x_{i+1} - x_i$ and $m_i = S''(x_i)$ be the unknown second derivative at each knot. Enforcing continuity of S, S', and S'' yields:

$$h_i\, m_{i-1} + 2(h_i + h_{i+1})\, m_i + h_{i+1}\, m_{i+1} = 6\left(\frac{y_{i+1} - y_i}{h_{i+1}} - \frac{y_i - y_{i-1}}{h_i}\right) \tag{3}$$

With natural boundary conditions $m_0 = m_N = 0$, this gives an $(N-1)\times(N-1)$ tridiagonal system solvable in $\mathcal{O}(N)$ — far cheaper than $\mathcal{O}(N^3)$ Gaussian elimination.

Recommended Reading

- *Numerical Recipes*, W.H. Press et al. — Ch. 1 (Preliminaries on Errors), Ch. 5.3 (Polynomial Interpolation), Ch. 5.7 (Numerical Derivatives).
- *Computational Physics*, Mark Newman — Ch. 4 (Accuracy and Speed), Ch. 5 (Integrals and Derivatives).

2 Lecture 02 — Numerical Integration, Root Finding, and Linear Equations

Date: 12/04/2026

2.1 Numerical Integration (Quadrature)

Numerical integration calculates the area under a curve $\int_a^b f(x)\,dx$ by evaluating the function at specific discrete points.

- **Simpson's Rule**: Groups points in sets of three and fits a parabola through them. The area under this parabola gives:

$$\int_{-h}^{h} f(x)\,dx = \frac{h}{3}\big[f(-h) + 4f(0) + f(h)\big] \tag{4}$$

  It is remarkably accurate, possessing a global error of $\mathcal{O}(h^4)$.

- **Gauss-Legendre Integration**: Optimises both the weights w_i and sampling locations x_i simultaneously, doubling the degrees of freedom. The optimal x_i match the roots of the Legendre polynomials. Using n optimally placed points, Gauss-Legendre quadrature perfectly integrates any polynomial up to degree $2n-1$.

- **Recursive (Adaptive) Integration**: Evaluates the integral on a coarse interval, splits it in half, and re-evaluates. If the two answers agree within tolerance ε, it stops; otherwise it recursively dives deeper only in highly-varying sub-regions.

2.2 Linear Equations (Gaussian Elimination)

Solving a system of n linear equations $Ax = b$ is the backbone of almost all computational physics. **Gaussian Elimination** systematically subtracts rows to transform A into an upper-triangular matrix in $\mathcal{O}(n^3)$ operations, then solves by **Back Substitution** in $\mathcal{O}(n^2)$.

The Critical Necessity of Partial Pivoting

During Gaussian elimination, we divide the entire row by the pivot element A_{ii}. If the pivot is very small, say 10^{-10}, dividing amplifies roundoff errors by a factor of 10^{10}, destroying the rest of the matrix.
Partial Pivoting: Before processing each column, scan down the rows to find the element with the largest absolute magnitude and swap it to the top. By always dividing by the largest possible number, we suppress error amplification and guarantee numerical stability.

Recommended Reading

- *Numerical Recipes*, W.H. Press et al. — Ch. 4 (Integration of Functions), Ch. 2.2 (Gaussian Elimination with Backsubstitution).
- *Computational Physics*, Mark Newman — Ch. 5 (Integrals and Derivatives), Ch. 6 (Solution of Linear and Nonlinear Equations).

3 Lecture 03 — Band Matrices, Least Squares, Gram-Schmidt, and Eigenvalues

Date: 23/04/2026

3.1 Large and Sparse Matrices (Band Matrices)

In real physical systems, "everything does not interact with everything." Consider the 1D heat diffusion equation on a wire discretised into 10,000 segments. The temperature at segment i only directly influences its immediate neighbours $i-1$ and $i+1$. The interaction matrix A is $10{,}000 \times 10{,}000$ (100 million elements), but 99.99% are zero — only the main diagonal and two adjacent diagonals are non-zero. This is a **Band Matrix**.

By storing and operating only on the band of width n_b, Gaussian elimination time drops from $\mathcal{O}(n^3)$ to $\mathcal{O}(n_b^2\, n)$ and memory from gigabytes to megabytes.

3.2 Least Squares

When fitting a line to noisy experimental data, we have an overdetermined system $Ax = b$ with more equations (data points) than unknowns. A perfect solution is impossible.
Physical intuition: The columns of A span the "model space". The data vector b lives outside this space due to noise. The best parameters x are found by dropping a perpendicular from b onto the model space. The error vector $(b - Ax)$ must be orthogonal to every column of A:

$$A^T(b - Ax) = 0 \implies \boxed{A^T A\, x = A^T b}$$

These are the **Normal Equations**. Solving them minimises the squared error $\|Ax - b\|^2$.

3.3 Gram-Schmidt Orthogonalization

Explicitly forming $A^T A$ is numerically dangerous because it squares the condition number of the matrix, violently amplifying floating-point inaccuracies. Instead, **Gram-Schmidt** orthogonalises the columns of A directly.
Physical intuition: For each vector a_j, subtract its projection onto all previously established orthogonal basis vectors:

$$a'_j = a_j - \frac{a_1 \cdot a_j}{a_1 \cdot a_1}\, a_1 \tag{5}$$

Repeating for all j yields mutually orthogonal columns.

Connection to QR decomposition. Gram-Schmidt is algebraically equivalent to a QR factorisation $A = \hat{A}R$, where $\hat{A}$ has orthonormal columns ($\hat{A}^T\hat{A} = I$). Substituting into the Normal Equations:

$$A^T A\, x = A^T b \implies Rx = \hat{A}^T b \tag{6}$$

This upper-triangular system is solved by back-substitution in $\mathcal{O}(m^2)$, entirely bypassing formation of $A^T A$.

Numerical Stability Warning: Loss of Orthogonality

If the columns of A are nearly linearly dependent, the remainder after subtraction can be many orders of magnitude smaller than the original. Once intermediate values approach machine epsilon ($\varepsilon_{\text{mach}} \approx 10^{-15}$), the subtraction is dominated by floating-point noise and **orthogonality is silently lost**. In practice, use *Modified Gram-Schmidt* (subtracting projections one at a time) or Householder reflections.

3.4 Introduction to Eigenvalues — Jacobi Method

For a real symmetric matrix A, we seek λ_i such that $A\varphi_i = \lambda_i\varphi_i$. The **Jacobi Method** applies a sequence of plane-rotation transformations T^TAT, where each T is a Givens rotation that zeros one off-diagonal pair (A_{pq}, A_{qp}).
Physical intuition: At each step, take the 2D slice of the N-dimensional matrix containing the largest off-diagonal element and physically rotate the coordinate system in that plane until the element vanishes. Convergence is guaranteed by two invariants:

1. **Trace Invariance**: $\mathrm{Tr}(T^TAT) = \mathrm{Tr}(A)$. The sum of eigenvalues never changes.
2. **Frobenius Norm Invariance**: $\sum_{i,j}(T^TAT)^2_{ij} = \sum_{i,j} A^2_{ij}$. Total squared weight is conserved under orthonormal transformations.

Why the Algorithm Must Converge

The Frobenius norm is shared between diagonal and off-diagonal entries. Since the total is fixed, weight removed from off-diagonal must go to the diagonal. After each full cycle of $n(n-1)/2$ rotations:

$$S_{\text{off}}^{(k+1)} \leq S_{\text{off}}^{(k)} \cdot e^{-1} \tag{7}$$

so S_{off} decreases by at least e^{-1} per cycle and convergence to a diagonal matrix is guaranteed.

Recommended Reading

- *Numerical Recipes*, W.H. Press et al. — Ch. 2.4 (Band Diagonal Systems), Ch. 15.4 (General Linear Least Squares), Ch. 11.0 (Eigensystems Introduction).
- *An Introduction to Computational Physics*, Tao Pang — Ch. 4 (Matrix Operations).

4 Lecture 04 — Jacobi Convergence, Multidimensional Roots, and Minimization

Date: 26/04/2026

4.1 Jacobi Method: Convergence Rate and Complexity

Convergence Rate per Cycle. Define a **cycle** as one complete pass through all $\frac{n(n-1)}{2}$ off-diagonal element pairs. Let $S^{(k)} = \sum_{i \neq j} A_{ij}^2$ after k cycles. Applying the per-rotation bound repeatedly over a full cycle:

$$S^{(1)} \leq S^{(0)} \cdot \left(1 - \frac{2}{n(n-1)}\right)^{n(n-1)/2} \approx S^{(0)} \cdot e^{-1} \tag{8}$$

Each cycle reduces the off-diagonal weight by $\approx e^{-1} \approx 0.37$. In practice, 10 cycles is sufficient for nearly all applications.

Computational Complexity. Each rotation modifies 2 rows and 2 columns of n elements, requiring ~ 12 multiplications per zero created. A full cycle has $\frac{n(n-1)}{2}$ rotations; for 10 cycles the total is $\approx 60n^3$, confirming $\mathcal{O}(n^3)$ scaling.

Tracking Eigenvectors. Maintain the accumulated rotation matrix $T = T_1 T_2 \cdots T_k$ (initialised as $T = I$). After each plane rotation with $\cos\phi = c$, $\sin\phi = s$ on the (p, q) plane, update:

$$T_{r,p} \leftarrow c\, T_{r,p} - s\, T_{r,q} \tag{9}$$

$$T_{r,q} \leftarrow s\, T_{r,p} + c\, T_{r,q} \tag{10}$$

At convergence the columns of T are the eigenvectors φ_i.

Optimisation: Row-Maximum Vector ($\mathcal{O}(n^4) \to \mathcal{O}(n^3)$)

Naive bottleneck: scanning the full upper-triangular part to find $\max_{i<j} |A_{ij}|$ costs $\mathcal{O}(n^2)$ per rotation, giving total $\mathcal{O}(n^4)$.
Fix: maintain a vector `j_max`$[i]$ storing the column index of the largest off-diagonal element in row i. Finding the global max then costs $\mathcal{O}(n)$. After each rotation (which modifies rows and columns p and q), only rows p, q, and rows that had their max in $\{p, q\}$ need re-scanning — costing $\mathcal{O}(n)$ expected total per rotation. Net result: $\mathcal{O}(n^3)$ total.

4.2 Finding the Root of a Function in Multiple Dimensions

Physical intuition: In 1D, bracketing (bisection) works because a sign change guarantees a root. In N dimensions, the zero-set of each component $F_i(\mathbf{x}) = 0$ is a hypersurface; roots are intersections of N such hypersurfaces. There is no bounding box whose corner values can certify a root lies inside — we must use local iterative methods that depend sensitively on the starting point.

Multidimensional Newton-Raphson. Construct the flat tangent hyperplane at the current point using the **Jacobian matrix** $J_{ij} = \frac{\partial F_i}{\partial x_j}$. The Newton step δ satisfies:

$$\boxed{J\,\delta = -F(\mathbf{x})} \tag{11}$$

This is solved by Gaussian elimination (Lecture 2). The new estimate is $\mathbf{x}_{\text{new}} = \mathbf{x} + \delta$, and the process repeats until $|F(\mathbf{x})|$ is acceptably small. Convergence is *quadratic* near a root (correct digits double each iteration), but a poor starting point can diverge.

4.3 Line Search and Backtracking

The Blind Hiker Analogy

A blindfolded hiker finds the downhill direction via Newton's method but taking a massive step might overshoot the valley bottom and ascend the opposite face. Solution: define the scalar merit function $f(\mathbf{x}) = \frac{1}{2}F(\mathbf{x}) \cdot F(\mathbf{x})$ and take a step λp. If $f(\mathbf{x}_{\text{new}}) > f(\mathbf{x})$, backtrack by reducing λ using parabolic/cubic curve fitting. Every step must strictly lower f.

Descent Guarantee. With the Newton direction $p = -J^{-1}F$:

$$\nabla f \cdot p = F^T J \cdot (-J^{-1}F) = -F \cdot F < 0 \tag{12}$$

so p is always a descent direction. We accept λ once the **Armijo condition** is satisfied:

$$f(\mathbf{x} + \lambda p) \leq f(\mathbf{x}) + \varepsilon \nabla f(\mathbf{x}) \cdot (\lambda p), \qquad \varepsilon \approx 10^{-4} \tag{13}$$

Backtracking Steps.

Parabolic and Cubic Polynomial Fitting

First backtrack — parabolic fit. From $g(0)$, $g'(0) < 0$, and $g(1)$, the parabola minimum is at:

$$\lambda^* = -\frac{g'(0)}{2\big[g(1) - g(0) - g'(0)\big]} \tag{14}$$

Clamped to $[0.1, 0.5]$ to prevent steps that are negligibly small or dangerously large.
Subsequent backtracks — cubic fit. With four known values $g(0)$, $g'(0)$, $g(\lambda_1)$, $g(\lambda_2)$, solve for the cubic minimum:

$$\frac{1}{\lambda_1 - \lambda_2} \begin{pmatrix} \lambda_1^{-2} & -\lambda_2^{-2} \\ -\lambda_1^{-3} & \lambda_2^{-3} \end{pmatrix} \begin{pmatrix} g(\lambda_1) - g(0) - g'(0)\lambda_1 \\ g(\lambda_2) - g(0) - g'(0)\lambda_2 \end{pmatrix} = \begin{pmatrix} a \\ b \end{pmatrix} \tag{15}$$

Minimum at $\lambda^* = \big(-b + \sqrt{b^2 - 3a\, g'(0)}\big)/(3a)$, again clamped.

4.4 Finding a Minimum in One Dimension — Golden Section Search

To bracket the minimum of a 1D function without derivatives, maintain three points $a < b < c$. Evaluate a new point x inside the larger interval to shrink the bracket. The mathematically optimal placement guarantees equal fractional reduction regardless of outcome, forcing the intervals to obey the **Golden Ratio**:

$$w = \frac{3 - \sqrt{5}}{2} \approx 0.382 \tag{16}$$

By always choosing x exactly 38.2% into the larger segment, the bracket shrinks optimally at every step.

4.5 Minimization in a Multidimensional Space

When finding a multidimensional minimum, gradient descent leads to a flat spot ($\nabla f = 0$). To confirm it is a minimum (not a saddle point), the landscape must curve upward in *every* direction: the **Hessian Matrix** $H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$ must be **Positive Definite**.
Efficient check: Apply Gaussian elimination to H. If and only if every diagonal pivot $B_{ii} > 0$ during elimination, the matrix is positive definite, confirming a true minimum. (Computing all eigenvalues would cost $\mathcal{O}(n^3)$ with a large prefactor; this check is cheaper and equally conclusive.)

Recommended Reading

- *Numerical Recipes*, W.H. Press et al. — Ch. 11.1 (Jacobi Transformations), Ch. 9.7 (Globally Convergent Methods for Nonlinear Systems), Ch. 10.1 (Golden Section Search).
- *Computational Physics*, Mark Newman — Ch. 6 (Solution of Linear and Nonlinear Equations).

5 Lecture 05 — Gradient Descent, Newton's Optimization Method, and Introduction to ODEs

Date: 30/04/2026

Overview

This lecture transitions from root-finding (Lecture 4) to *optimization* — finding a point where the gradient vanishes — and then opens the topic of *numerical integration of ordinary differential equations*. The two topics are related: the gradient-descent step is itself governed by a continuous-time ODE, and the Euler method we derive here is the simplest possible discretisation of any such ODE.

5.1 Gradient Descent

Physical intuition: Imagine a blindfolded hiker on a multidimensional landscape trying to reach the lowest valley. At each step the hiker measures the slope in every direction (the gradient ∇f) and steps in the *opposite* direction — directly downhill. This is gradient descent.

5.1.1 Derivation via Taylor Expansion

Let $f : \mathbb{R}^N \to \mathbb{R}$ be the objective function. Expand to first order around the current point $\mathbf{x}_0$:

$$f(\mathbf{x}_0 + \delta) \approx f(\mathbf{x}_0) + (\nabla f) \cdot \delta \tag{17}$$

To decrease f as rapidly as possible, choose δ antiparallel to the gradient:

$$\boxed{\delta = -\alpha \, \nabla f(\mathbf{x}_0)} \tag{18}$$

where $\alpha > 0$ is the **step size** (learning rate). The update rule is therefore:

$$\mathbf{x}_{n+1} = \mathbf{x}_n - \alpha \, \nabla f(\mathbf{x}_n) \tag{19}$$

5.1.2 Step Size Selection via 1D Line Search

Choosing α is non-trivial:

- Too large: the update overshoots the minimum and f may increase.
- Too small: convergence is extremely slow.

Optimal strategy: treat α as a 1D optimisation problem. Define $g(\alpha) = f(\mathbf{x}_n - \alpha \, \nabla f)$ and minimise it using Golden Section Search (Lecture 4, §4.3). This *exact line search* is expensive but guarantees the best step along the descent direction. In practice, the *backtracking* (Armijo) criterion from Lecture 4 is used instead — it is much cheaper and still ensures strict descent.

5.1.3 Convergence Properties and Limitations

- Gradient descent converges to a **local** minimum, not necessarily the global one. The algorithm has no mechanism to escape a local basin.
- Near the minimum, convergence is *linear* (error decreases by a constant factor per step). In contrast to Newton's method (below), which is quadratic, gradient descent can require many more steps in poorly scaled or ill-conditioned problems.
- The method is inefficient in narrow, elongated valleys (ravines): the gradient oscillates across the valley rather than proceeding along it.

5.2 Newton's Method for Optimization (Second-Order Methods)

Physical intuition: Gradient descent uses only the slope (first derivative) of the landscape. Newton's method adds the *curvature* (second derivative) — it fits a local paraboloid to the landscape and jumps directly to its bottom, rather than crawling downhill.

5.2.1 Second-Order Taylor Expansion and the Hessian

Expand f to second order around $\mathbf{x}_0$:

$$f(\mathbf{x}_0 + \delta) \approx f(\mathbf{x}_0) + (\nabla f) \cdot \delta + \tfrac{1}{2}\, \delta^T \mathbf{H}\, \delta \tag{20}$$

where the **Hessian matrix H** has entries

$$H_{ij} = \frac{\partial^2 f}{\partial x_i\, \partial x_j} \tag{21}$$

The local paraboloid has a unique minimum where its gradient vanishes:

$$\nabla_\delta \left[(\nabla f) \cdot \delta + \tfrac{1}{2}\, \delta^T \mathbf{H}\, \delta \right] = \nabla f + \mathbf{H}\, \delta = 0 \tag{22}$$

This gives the **Newton step**:

$$\boxed{\mathbf{H}\, \delta = -\nabla f} \tag{23}$$

5.2.2 Solving the Newton System

The equation $\mathbf{H}\, \delta = -\nabla f$ is a square linear system of size $N \times N$, solved by Gaussian elimination in $\mathcal{O}(N^3)$ operations — exactly the same infrastructure as multidimensional Newton-Raphson for roots (Lecture 4).

The new iterate is $\mathbf{x}_{\text{new}} = \mathbf{x} + \delta$. Near the minimum, convergence is *quadratic*: the number of correct digits roughly doubles each iteration.

5.2.3 Positive Definiteness Condition

For the paraboloid to have a minimum (not a maximum or saddle point), the Hessian must be **Positive Definite** (PD): all eigenvalues $\lambda_i > 0$, equivalently $\mathbf{v}^T \mathbf{H}\, \mathbf{v} > 0$ for all non-zero $\mathbf{v}$.

If $\mathbf{H}$ is not PD at the current point, the Newton step points toward a saddle or maximum — we are not near a minimum. In this case, one falls back to gradient descent or adds a positive multiple of the identity matrix (*Levenberg-Marquardt regularisation*) to make $\mathbf{H} + \mu I$ PD.

5.3 Checking Positive Definiteness via Gaussian Elimination

Physical intuition: Computing all N eigenvalues of $\mathbf{H}$ to verify positivity costs $\mathcal{O}(N^3)$ with a large prefactor. Gaussian elimination on a symmetric matrix offers an equally conclusive check at lower cost.

5.3.1 The Criterion

Apply Gaussian elimination (without row swaps, since $\mathbf{H}$ is symmetric) to $\mathbf{H}$. At each step, divide the k-th row by the diagonal pivot B_{kk}. The key theorem is:

Positive Definiteness via Gaussian Elimination

A real symmetric matrix $\mathbf{H}$ is Positive Definite *if and only if* all diagonal pivot elements B_{kk} remain strictly positive during Gaussian elimination without pivoting.
Why: The pivots $B_{11}, B_{22}, \ldots, B_{NN}$ are the ratios of successive leading principal minors:

$$B_{kk} = \frac{\det(\mathbf{H}_k)}{\det(\mathbf{H}_{k-1})}$$

By Sylvester's criterion, $\mathbf{H}$ is PD $\iff$ all leading principal minors are positive $\iff$ all pivots are positive. If any $B_{kk} \leq 0$, the matrix is not PD and we are not at a (local) minimum.

5.3.2 Connection to the Minimization Check (Lecture 4)

In Lecture 4 we noted that checking positive definiteness of $\mathbf{H}$ via Gaussian elimination is cheaper than computing eigenvalues. The current lecture provides the algorithmic detail: run standard Gaussian elimination on $\mathbf{H}$ (which is symmetric, so only the upper triangle need be stored) and inspect the diagonal pivots. Cost: $\mathcal{O}(N^3/6)$, a factor of 6 cheaper than full LU with the same asymptotic scaling.

5.4 Ordinary Differential Equations: Problem Setup

Physical intuition: Almost all of classical physics is expressed as ODEs: Newton's second law, circuit equations, orbital mechanics. The goal is to integrate these equations forward in time to find the trajectory $y(x)$ given an initial condition.

5.4.1 Standard Form: First-Order System

The general first-order ODE is:

$$\frac{dy}{dx} = f\big(x,\, y(x)\big), \qquad y(x_0) = y_0 \tag{24}$$

Higher-order equations are reduced to this form by introducing auxiliary variables. Consider Newton's second law:

$$m\,\ddot{r} = F(t, r, \dot{r}) \tag{25}$$

Define $v = \dot{r}$. This becomes a *system* of two first-order ODEs:

$$\dot{r} = v \tag{26}$$

$$\dot{v} = \frac{F(t, r, v)}{m} \tag{27}$$

In vector form $\mathbf{y} = (r, v)^T$:

$$\frac{d\mathbf{y}}{dt} = \mathbf{f}(t, \mathbf{y}) \tag{28}$$

All standard numerical ODE solvers work on this canonical first-order system form, so the reduction is not merely a notational convenience — it is the required input format.

5.5 Euler's Method

Physical intuition: Euler's method is the simplest possible numerical ODE integrator. At each step, assume the derivative $f(x_n, y_n)$ is constant over the interval $[x_n, x_{n+1}]$ and use it to extrapolate linearly.

5.5.1 Derivation

Discretise the domain into uniform steps of size h: $x_n = x_0 + n\,h$. Replace the derivative by its forward-difference approximation:

$$\frac{y_{n+1} - y_n}{h} \approx f(x_n, y_n) \tag{29}$$

Solving for y_{n+1}:

$$\boxed{y_{n+1} = y_n + h\,f(x_n,\, y_n)} \tag{30}$$

5.5.2 Error Analysis

Local (per-step) error. Expand the exact solution in Taylor series:

$$y(x_{n+1}) = y(x_n) + h\,y'(x_n) + \frac{h^2}{2}\,y''(x_n) + \mathcal{O}\big(h^3\big) \tag{31}$$

Since $y'(x_n) = f(x_n, y_n)$, the Euler update matches the first two terms. The local truncation error is therefore:

$$e_{\text{local}} = \frac{h^2}{2}\,y''(x_n) + \mathcal{O}\big(h^3\big) = \mathcal{O}\big(h^2\big) \tag{32}$$

Global error. Over the full domain $[x_0, X]$ there are $n = (X - x_0)/h$ steps, so errors accumulate:

$$e_{\text{global}} \approx n \cdot e_{\text{local}} = \frac{X - x_0}{h} \cdot \mathcal{O}\big(h^2\big) = \mathcal{O}(h) \tag{33}$$

Euler's method is therefore a **first-order** method: halving the step size halves the global error.

Why Euler's Method is Rarely Used in Practice

Although simple and instructive, Euler's method suffers from two problems:

1. **Accuracy**: Only first-order global accuracy. To achieve 10^{-6} absolute error over unit interval requires $h \sim 10^{-6}$, i.e., one million evaluations of f.
2. **Stability**: For stiff equations (where f has a large Lipschitz constant), the step size must satisfy $h < 2/|f_y|$ or the error grows exponentially. This can force h to be astronomically small even when the solution itself is smooth.

Higher-order methods (Runge-Kutta, Adams-Bashforth) achieve much better accuracy per function evaluation. The Runge-Kutta 4 (RK4) method, covered in the next lecture, achieves $\mathcal{O}(h^4)$ local error ($\mathcal{O}(h^4)$ global) with only 4 evaluations of f per step.

Recommended Reading

- *Numerical Recipes*, W.H. Press et al. — Ch. 10.6 (Gradient Methods; Conjugate Gradients), Ch. 16.1 (Runge-Kutta Method; Euler as special case).
- *Computational Physics*, Mark Newman — Ch. 8 (Ordinary Differential Equations).

- *An Introduction to Computational Physics*, Tao Pang — Ch. 3 (Ordinary Differential Equations).

6 Lecture 06 — Higher-Order ODE Methods, Adaptive Steps, and Stability

Date: 03/05/2026

Overview

Lecture 5 introduced Euler's method as the simplest numerical integrator for a first-order ordinary differential equation $y'(x) = f(x, y)$. Euler's method is useful pedagogically, but it is rarely efficient: its local truncation error is only $\mathcal{O}(h^2)$ and its global error is only $\mathcal{O}(h)$. This lecture develops two systematic routes to higher accuracy:

- **multi-step methods**, which reuse previously computed values of f; and
- **Runge-Kutta methods**, which compute several slopes inside the current step.

The lecture then explains adaptive step-size control and closes with the central stability lesson: explicit methods can become unstable even when the exact solution is perfectly well behaved, while implicit and semi-implicit methods often improve stability at the cost of solving algebraic equations.

6.1 Taylor Expansion Beyond Euler

Physical Motivation

Euler's method assumes that the slope measured at the beginning of the step remains constant throughout the whole interval $[x_n, x_{n+1}]$. If the true slope changes noticeably during the step, this straight-line extrapolation misses the curvature of the solution. The most direct repair is therefore to keep more terms in the Taylor expansion of the exact solution.

Let $h = x_{n+1} - x_n$ be the step size, y_n the numerical approximation to $y(x_n)$, and

$$y'(x) = f(x, y(x)). \tag{34}$$

Expanding the exact solution around x_n gives

$$y(x_{n+1}) = y(x_n) + h\, y'(x_n) + \frac{h^2}{2}\, y''(x_n) + \mathcal{O}\big(h^3\big)\,. \tag{35}$$

The first derivative is fixed by the differential equation:

$$y'(x_n) = f(x_n, y_n). \tag{36}$$

For the second derivative we must differentiate $f(x, y(x))$ along the solution curve. By the chain rule,

$$y''(x) = \frac{d}{dx} f(x, y(x)) = \frac{\partial f}{\partial x}(x, y) + \frac{\partial f}{\partial y}(x, y)\, \frac{dy}{dx}. \tag{37}$$

Using again $dy/dx = f(x, y)$,

$$y''(x) = f_x(x, y) + f_y(x, y)\, f(x, y), \tag{38}$$

where $f_x = \partial f/\partial x$ and $f_y = \partial f/\partial y$. Therefore a second-order Taylor method is

$$\boxed{y_{n+1} = y_n + h\, f_n + \frac{h^2}{2}\,(f_{x,n} + f_{y,n} f_n)} \tag{39}$$

with $f_n = f(x_n, y_n)$.

Physical significance: the Taylor method improves Euler by adding the curvature of the trajectory, but it requires derivatives of f itself, which may be unavailable or expensive in realistic simulations.

6.2 Adams-Bashforth Explicit Multi-Step Methods

Physical Motivation

Instead of differentiating f, we can reuse information already collected while integrating. If f was known at x_{n-1} and x_n, then the change of slope between those points gives a cheap estimate of how f will behave over the next step. This is useful when evaluating f is costly but stored past values are essentially free.

The exact integral form of the ODE is

$$y_{n+1} = y_n + \int_{x_n}^{x_{n+1}} f(x, y(x))\, dx. \tag{40}$$

The Adams-Bashforth idea is to approximate the integrand over $[x_n, x_{n+1}]$ by extrapolating from earlier values

$$f_j = f(x_j, y_j), \qquad x_j = x_0 + jh. \tag{41}$$

6.2.1 Two-Step Adams-Bashforth Method

Use a linear extrapolation through (x_{n-1}, f_{n-1}) and (x_n, f_n):

$$f(x) \approx f_n + \frac{x - x_n}{h}\,(f_n - f_{n-1})\,. \tag{42}$$

Substitute this into Eq. (40). The first term gives

$$\int_{x_n}^{x_{n+1}} f_n\, dx = h f_n, \tag{43}$$

and the correction term gives

$$\int_{x_n}^{x_{n+1}} \frac{x - x_n}{h}\,(f_n - f_{n-1})\; dx = \frac{h}{2}\,(f_n - f_{n-1})\,. \tag{44}$$

Therefore

$$\boxed{y_{n+1} = y_n + h\left(\frac{3}{2} f_n - \frac{1}{2} f_{n-1}\right)} \tag{45}$$

and the local truncation error is $\mathcal{O}(h^3)$.

Physical significance: Adams-Bashforth gains one order over Euler without evaluating f at a new intermediate point; it pays for this by requiring one previous step.

6.2.2 Four-Step Adams-Bashforth Method

Using more past points means fitting a higher-degree interpolating polynomial to the old values of f and integrating that polynomial. With four past values the standard formula is

$$\boxed{y_{n+1} = y_n + \frac{h}{24}\,(55 f_n - 59 f_{n-1} + 37 f_{n-2} - 9 f_{n-3})} \tag{46}$$

with local truncation error $\mathcal{O}(h^5)$.

Physical significance: high-order Adams-Bashforth methods can be very efficient for smooth problems with constant step size, because after the startup phase each new step needs only one fresh evaluation of f.

The price is that a k-step method cannot start by itself. For example, Eq. (46) needs $f_n, f_{n-1}, f_{n-2}, f_{n-3}$. The first few values must be generated by another method, commonly Euler with very small steps or a Runge-Kutta method.

6.3 Implicit Adams Methods

Physical Motivation

The Adams-Bashforth methods look only backward. A more balanced estimate of the integral also uses information at the end of the interval, x_{n+1}. This usually improves accuracy and stability, but it creates an implicit equation because the unknown y_{n+1} appears inside $f(x_{n+1}, y_{n+1})$.

The simplest implicit formula averages the slope at the beginning and at the end of the step:

$$\boxed{y_{n+1} = y_n + \frac{h}{2}\left(f_n + f_{n+1}\right)} \tag{47}$$

where $f_{n+1} = f(x_{n+1}, y_{n+1})$. This is the trapezoidal method and has local truncation error $\mathcal{O}(h^3)$.

Physical significance: using the future slope makes the step less biased toward the beginning of the interval, which improves accuracy and often gives better stability.

For a linear equation of the form

$$y' = g(x)y, \tag{48}$$

Eq. (47) can be solved explicitly. We write

$$y_{n+1} = y_n + \frac{h}{2}\left(g(x_n)y_n + g(x_{n+1})y_{n+1}\right). \tag{49}$$

Move the unknown term to the left:

$$\left(1 - \frac{h}{2}g(x_{n+1})\right) y_{n+1} = \left(1 + \frac{h}{2}g(x_n)\right) y_n. \tag{50}$$

Thus

$$\boxed{y_{n+1} = \frac{1 + \frac{h}{2}g(x_n)}{1 - \frac{h}{2}g(x_{n+1})}\, y_n} \tag{51}$$

Physical significance: for linear problems, an implicit step can often be rewritten as a simple algebraic update; for nonlinear problems it usually requires a root-finding procedure such as Newton-Raphson.

A higher-order implicit Adams-Moulton formula using f_{n+1}, f_n, f_{n-1} is

$$\boxed{y_{n+1} = y_n + \frac{h}{12}\left(5f_{n+1} + 8f_n - f_{n-1}\right)} \tag{52}$$

with local truncation error $\mathcal{O}(h^4)$. With one more previous point,

$$\boxed{y_{n+1} = y_n + \frac{h}{24}\left(9f_{n+1} + 19f_n - 5f_{n-1} + f_{n-2}\right)} \tag{53}$$

with local truncation error $\mathcal{O}(h^5)$.

Physical significance: Adams-Moulton methods are accurate and stable, but because they are implicit they are most convenient when paired with a predictor.

6.3.1 Predictor-Corrector Use

A practical compromise is:

1. predict y^*_{n+1} with an explicit Adams-Bashforth formula;
2. evaluate $f^*_{n+1} = f(x_{n+1}, y^*_{n+1})$;
3. correct the result with an Adams-Moulton formula.

For example, Eq. (46) can predict the endpoint, and Eq. (53) can correct it:

$$\boxed{y_{n+1} = y_n + \frac{h}{24}\left(9f^*_{n+1} + 19f_n - 5f_{n-1} + f_{n-2}\right)} \tag{54}$$

Physical significance: predictor-corrector methods keep most of the low cost of explicit multi-step integration while importing some of the accuracy and stability advantages of implicit formulas.

6.4 Runge-Kutta Methods

Physical Motivation

Multi-step methods assume that the previously sampled values of f remain a good guide to the next interval. This works well for smooth solutions with constant h, but it is awkward when the step size must change or when the slope varies sharply in a localized region. Runge-Kutta methods solve this by sampling several slopes inside the current step only.

6.4.1 Second-Order Midpoint Method

Start with the slope at the beginning:

$$k_1 = h\, f(x_n, y_n). \tag{55}$$

Use it only to estimate the midpoint value:

$$y\left(x_n + \frac{h}{2}\right) \approx y_n + \frac{k_1}{2}. \tag{56}$$

Now evaluate the slope at that midpoint:

$$k_2 = h\, f\left(x_n + \frac{h}{2},\, y_n + \frac{k_1}{2}\right). \tag{57}$$

The update is

$$\boxed{y_{n+1} = y_n + k_2} \tag{58}$$

with local truncation error $\mathcal{O}(h^3)$.

Physical significance: RK2 replaces Euler's beginning-of-step slope by a midpoint slope, so it captures the leading curvature of the trajectory without needing analytic derivatives of f.

6.4.2 Third-Order Runge-Kutta from Simpson's Rule

Simpson's rule suggests weighting the beginning, middle, and end of the interval as $1 : 4 : 1$. One third-order Runge-Kutta construction is

$$k_1 = h\, f(x_n, y_n), \tag{59}$$

$$k_2 = h\, f\left(x_n + \frac{h}{2},\, y_n + \frac{k_1}{2}\right), \tag{60}$$

$$k_3 = h\, f\left(x_n + h,\, y_n + 2k_2 - k_1\right). \tag{61}$$

The update is

$$\boxed{y_{n+1} = y_n + \frac{1}{6}\left(k_1 + 4k_2 + k_3\right)} \tag{62}$$

with local truncation error $\mathcal{O}(h^4)$.

Physical significance: RK3 uses three internally sampled slopes to imitate a high-order quadrature rule for the integral of f over the step.

6.4.3 Classical Fourth-Order Runge-Kutta

The most widely used basic ODE solver in computational physics is classical RK4:

$$k_1 = h\, f(x_n, y_n), \tag{63}$$

$$k_2 = h\, f\left(x_n + \frac{h}{2},\, y_n + \frac{k_1}{2}\right), \tag{64}$$

$$k_3 = h\, f\left(x_n + \frac{h}{2},\, y_n + \frac{k_2}{2}\right), \tag{65}$$

$$k_4 = h\, f(x_n + h,\, y_n + k_3). \tag{66}$$

Then

$$\boxed{y_{n+1} = y_n + \frac{1}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right)} \tag{67}$$

with local truncation error $\mathcal{O}(h^5)$ and global error $\mathcal{O}(h^4)$.

Physical significance: RK4 is often the best first choice for smooth non-stiff ODEs: it is accurate, self-starting, and does not require storing old steps.

6.5 Adaptive Step-Size Control

Physical Motivation

A fixed step size wastes work in calm regions and may fail in rapidly changing regions. A good integrator should take large steps when the solution is smooth and automatically shrink the step when the local error estimate becomes too large.

Suppose we integrate from a to b and want a total allowed absolute error ε. During one attempted step of size h, compute two approximations:

- y_1: one step of size h;
- y_2: two steps of size $h/2$.

The difference

$$E_h = |y_2 - y_1| \tag{68}$$

is used as a practical local error estimate. A natural tolerance for this step is

$$\delta_h = \varepsilon \frac{h}{b-a}. \tag{69}$$

The basic decision rule is

$$\boxed{E_h < \delta_h \quad \Longrightarrow \quad \text{accept the step}} \tag{70}$$

Physical significance: Eq. (70) distributes the total error budget approximately in proportion to the length of each accepted step.

If $E_h > \delta_h$, reject the step, reduce h (for example by a factor of 2), and try again. If E_h is safely below the tolerance, the next step can use a larger h; if it is close to the tolerance, the next step should use a smaller h. In vector problems, replace the absolute value in Eq. (68) by a chosen norm such as $\|\mathbf{y}_2 - \mathbf{y}_1\|$.

6.5.1 Embedded Runge-Kutta Estimates

The one-step-versus-two-half-steps test is reliable but expensive because it repeats the whole Runge-Kutta calculation. A more economical method computes two different order estimates from mostly the same internal slopes:

$$y_{n+1}^{(p)} = y_n + \sum_{i=1}^{s} b_i k_i, \qquad y_{n+1}^{(p-1)} = y_n + \sum_{i=1}^{s} \tilde{b}_i k_i. \tag{71}$$

The difference

$$E_h = \left| y_{n+1}^{(p)} - y_{n+1}^{(p-1)} \right| \tag{72}$$

estimates the local error without recomputing the whole step.

Physical significance: embedded Runge-Kutta methods turn the slopes already computed for the step into an error bar, which makes adaptive integration much cheaper.

6.5.2 Richardson Extrapolation from One and Two Half-Steps

Assume a method has leading local error proportional to h^3:

$$y_1 = y_{\text{exact}} + Ah^3 + Bh^4 + \cdots, \tag{73}$$

$$y_2 = y_{\text{exact}} + 2A\left(\frac{h}{2}\right)^3 + 2B\left(\frac{h}{2}\right)^4 + \cdots = y_{\text{exact}} + \frac{Ah^3}{4} + \frac{Bh^4}{8} + \cdots. \tag{74}$$

Choose a linear combination that cancels the Ah^3 term:

$$\boxed{y_{\text{ext}} = \frac{4}{3}y_2 - \frac{1}{3}y_1} \tag{75}$$

Physical significance: if the leading error behaves smoothly with h, Richardson extrapolation uses two imperfect approximations to construct a more accurate one.

6.6 Stability of Explicit and Implicit Euler Methods

Physical Motivation

Accuracy asks whether the numerical solution is close to the exact solution when h is small. Stability asks a different question: do small numerical errors decay or grow as the integration proceeds? A method can be formally accurate and still unusable if it amplifies errors step after step.

Consider the test equation

$$y' = -c\,y, \qquad c > 0, \qquad y(0) = y_0. \tag{76}$$

The exact solution is

$$y(x) = y_0 e^{-cx}, \tag{77}$$

so the physical behavior is monotone decay to zero.

6.6.1 Explicit Euler Stability

Explicit Euler gives

$$y_{n+1} = y_n + h(-cy_n) = (1 - ch)y_n. \tag{78}$$

After many steps,

$$y_n = (1 - ch)^n y_0. \tag{79}$$

For decay rather than growth, the amplification factor must satisfy

$$|1 - ch| < 1. \tag{80}$$

Since $c > 0$ and $h > 0$, this is equivalent to

$$\boxed{h < \frac{2}{c}} \tag{81}$$

Physical significance: even for the harmless equation $y' = -cy$, explicit Euler becomes unstable if the step is too large; the computed solution alternates sign and grows instead of decaying.

6.6.2 Implicit Euler Stability

Implicit Euler evaluates the slope at the unknown endpoint:

$$y_{n+1} = y_n + h(-cy_{n+1}). \tag{82}$$

Move the endpoint term to the left:

$$(1 + ch)y_{n+1} = y_n. \tag{83}$$

Therefore

$$\boxed{y_{n+1} = \frac{y_n}{1 + ch}} \tag{84}$$

Physical significance: the amplification factor $(1+ch)^{-1}$ is always between 0 and 1 for $c, h > 0$, so implicit Euler is stable for every positive step size on this decay problem.

6.6.3 Semi-Implicit Linearization

For a nonlinear autonomous equation

$$y' = f(y), \tag{85}$$

the fully implicit Euler step is

$$y_{n+1} = y_n + h\,f(y_{n+1}). \tag{86}$$

Solving Eq. (86) exactly may require Newton's method at every time step. A cheaper semi-implicit approximation linearizes f around y_n:

$$f(y_{n+1}) \approx f(y_n) + f'(y_n)(y_{n+1} - y_n). \tag{87}$$

Substitute into Eq. (86):

$$y_{n+1} = y_n + hf(y_n) + hf'(y_n)(y_{n+1} - y_n). \tag{88}$$

Collect terms proportional to $y_{n+1} - y_n$:

$$(1 - hf'(y_n))\,(y_{n+1} - y_n) = hf(y_n). \tag{89}$$

Thus

$$\boxed{y_{n+1} = y_n + h\,(1 - hf'(y_n))^{-1}\,f(y_n)} \tag{90}$$

Physical significance: semi-implicit Euler keeps part of the stability benefit of an implicit method while avoiding a full nonlinear solve at every step.

6.7 Systems of ODEs

Physical Motivation

Most physical ODE problems are systems: a particle has position and velocity, a circuit has several dynamical variables, and a discretized field has many degrees of freedom. The numerical methods above extend almost unchanged, but stability must be checked in all independent modes of the system.

For a vector unknown $\mathbf{y}(x) \in \mathbb{R}^N$, write

$$\frac{d\mathbf{y}}{dx} = \mathbf{f}(x, \mathbf{y}). \tag{91}$$

All explicit methods generalize by treating f and k_i as vectors. For example, RK4 becomes

$$\mathbf{k}_1 = h\,\mathbf{f}(x_n, \mathbf{y}_n), \tag{92}$$

$$\mathbf{k}_2 = h\,\mathbf{f}\left(x_n + \frac{h}{2}, \mathbf{y}_n + \frac{\mathbf{k}_1}{2}\right), \tag{93}$$

$$\mathbf{k}_3 = h\,\mathbf{f}\left(x_n + \frac{h}{2}, \mathbf{y}_n + \frac{\mathbf{k}_2}{2}\right), \tag{94}$$

$$\mathbf{k}_4 = h\,\mathbf{f}(x_n + h, \mathbf{y}_n + \mathbf{k}_3), \tag{95}$$

and

$$\boxed{\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{1}{6}\,(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)} \tag{96}$$

Physical significance: once the ODE is written as a first-order system, scalar algorithms apply component by component, with vector norms used for error control.

6.7.1 Linear Stability for Systems

Consider the linear decay system

$$\mathbf{y}' = -\mathbf{C}\mathbf{y}, \tag{97}$$

where $\mathbf{C}$ is positive definite. Explicit Euler gives

$$\mathbf{y}_{n+1} = (\mathbf{I} - h\mathbf{C})\,\mathbf{y}_n. \tag{98}$$

After many steps the repeated amplification matrix is

$$\mathbf{y}_n = (\mathbf{I} - h\mathbf{C})^n\,\mathbf{y}_0. \tag{99}$$

If $\lambda_{\max}$ is the largest eigenvalue of $\mathbf{C}$, stability requires

$$\boxed{h < \frac{2}{\lambda_{\max}}} \tag{100}$$

Physical significance: the fastest decaying mode of the physical system controls the allowed explicit step size; this is why stiff systems are difficult for explicit integrators.

Implicit Euler instead gives

$$(\mathbf{I} + h\mathbf{C})\,\mathbf{y}_{n+1} = \mathbf{y}_n, \tag{101}$$

or

$$\boxed{\mathbf{y}_{n+1} = (\mathbf{I} + h\mathbf{C})^{-1}\,\mathbf{y}_n} \tag{102}$$

Physical significance: the eigenvalues of $(\mathbf{I} + h\mathbf{C})^{-1}$ are $(1 + h\lambda_i)^{-1}$, which lie between 0 and 1 for $h > 0$ and $\lambda_i > 0$; the implicit system is therefore stable for all positive step sizes.

For nonlinear systems, the semi-implicit scalar formula becomes a matrix formula using the Jacobian

$$\mathbf{J}(\mathbf{y}) = \frac{\partial \mathbf{f}}{\partial \mathbf{y}}. \tag{103}$$

The update is

$$\boxed{\mathbf{y}_{n+1} = \mathbf{y}_n + h\,(\mathbf{I} - h\mathbf{J}(\mathbf{y}_n))^{-1}\,\mathbf{f}(\mathbf{y}_n)} \tag{104}$$

Physical significance: in multiple dimensions, semi-implicit integration replaces division by a scalar with solving a linear system; this is more expensive than an explicit step but can be essential for stability.

Recommended Reading

- *Computational Physics*, Mark Newman — Ch. 8.1–8.4 (Euler, Runge-Kutta, adaptive step sizes, and higher-order ODE methods).
- *Numerical Recipes*, W.H. Press et al. — Ch. 17.1–17.2 (Runge-Kutta methods, adaptive steps, and embedded error estimates).
- Tao Pang, *An Introduction to Computational Physics* — Ch. 3.2–3.4 (ODE solvers and stability).
- MIT OpenCourseWare, Numerical Methods for ODEs — useful short review of Euler and Runge-Kutta ideas.

7 Lecture 07 — Stiff ODEs, Numerov Method, Boundary-Value Problems, and Relaxation

Date: 07/05/2026

Overview

Lecture 6 ended with the stability restriction for explicit methods applied to systems of ODEs. This lecture begins by turning that criterion into a concrete example of a *stiff* system: a system whose fast-decaying modes force a tiny numerical step even though the physically interesting solution evolves slowly. The lecture then introduces Numerov's method for second-order equations of the form $y''(x) = f(x)y(x) + g(x)$, and moves from initial-value problems to boundary-value problems. The final part explains shooting, shooting to a fitting point, and relaxation, with a white-dwarf structure problem as the main physical example.

7.1 A Stiff Two-Variable System

Physical Motivation

In many physical systems several processes act on the same variables but on very different time scales. A chemical abundance may have a very fast transient and a slow secular evolution; a mechanical system may contain a heavily damped mode and a slowly damped mode. Numerically, the fast mode may control stability even after it has become physically negligible.

Consider the linear system

$$u' = 998u + 1998v, \tag{105}$$
$$v' = -999u - 1999v, \tag{106}$$

with initial data

$$u(0) = 1, \qquad v(0) = 0. \tag{107}$$

Introduce new variables y and z by

$$u = 2y - z, \qquad v = -y + z. \tag{108}$$

Adding the two original equations gives

$$y' = u' + v' = -u - v. \tag{109}$$

Since $u + v = y$, this becomes

$$\boxed{y' = -y} \tag{110}$$

Physical significance: y is the slow mode; it decays on the order-one scale in x.

To isolate z, use $u + 2v = z$. Therefore

$$z' = u' + 2v'. \tag{111}$$

Substituting Eqs. (105)–(106) and then Eq. (108) gives

$$z' = (998u + 1998v) + 2(-999u - 1999v) \tag{112}$$
$$= -1000u - 2000v = -1000(u + 2v) = -1000z. \tag{113}$$

Thus

$$\boxed{z' = -1000z} \tag{114}$$

Physical significance: z is the fast transient; it decays on the scale 10^{-3} in x.

The initial data imply

$$y(0) = u(0) + v(0) = 1, \qquad z(0) = u(0) + 2v(0) = 1. \tag{115}$$

Therefore

$$y(x) = e^{-x}, \qquad z(x) = e^{-1000x}. \tag{116}$$

Returning to u and v,

$$\boxed{u(x) = 2e^{-x} - e^{-1000x}}, \tag{117}$$

$$\boxed{v(x) = -e^{-x} + e^{-1000x}}. \tag{118}$$

Physical significance: after a very short initial layer, the solution is dominated by e^{-x}, but an explicit integrator still has to respect the stability constraint imposed by the e^{-1000x} mode.

For an explicit method such as Euler, Lecture 6 showed that a decay mode $e^{-\lambda x}$ imposes roughly

$$h < \frac{2}{\lambda}. \tag{119}$$

Here the largest decay rate is $\lambda = 1000$, so

$$\boxed{h < 2 \times 10^{-3}} \tag{120}$$

Physical significance: the numerical method must take thousands of tiny steps per unit interval even though the surviving physical mode varies on a scale of order 1. This mismatch is the hallmark of stiffness.

Implicit or semi-implicit methods are designed precisely for this situation: they damp the fast stable modes without forcing the step size to resolve their decay in detail.

7.2 Numerov's Method for Second-Order Equations

Physical Motivation

Many physics problems reduce directly to a second-order equation rather than to an arbitrary first-order system. If the equation is linear in y, then we can exploit the special structure to obtain a very accurate three-point method with little extra work. This is the purpose of Numerov's method.

Consider

$$y''(x) = f(x)y(x) + g(x), \tag{121}$$

where f and g are known functions. Two common physical examples are:

$$\psi''(x) = -k^2(x)\psi(x), \tag{122}$$

$$\frac{1}{r^2}\frac{d}{dr}\left(r^2\frac{d\phi}{dr}\right) = -4\pi\rho(r). \tag{123}$$

In the radial Poisson problem, defining $u(r) = r\phi(r)$ converts the radial operator to a second derivative:

$$u''(r) = -4\pi r\rho(r). \tag{124}$$

This is a special case of Eq. (121).

Physical Motivation

The central finite-difference formula for y'' is second order. Numerov's method improves it by using the differential equation itself to cancel the leading fourth-derivative error. The result is a symmetric stencil involving $x-h$, x, and $x+h$.

Expand $y(x+h)$ and $y(x-h)$ around x:

$$y(x+h) = y + hy' + \frac{h^2}{2}y'' + \frac{h^3}{6}y''' + \frac{h^4}{24}y^{(4)} + \frac{h^5}{120}y^{(5)} + \mathcal{O}\left(h^6\right), \tag{125}$$

$$y(x-h) = y - hy' + \frac{h^2}{2}y'' - \frac{h^3}{6}y''' + \frac{h^4}{24}y^{(4)} - \frac{h^5}{120}y^{(5)} + \mathcal{O}\left(h^6\right). \tag{126}$$

Adding them cancels the odd derivatives:

$$y(x+h) + y(x-h) = 2y + h^2y'' + \frac{h^4}{12}y^{(4)} + \mathcal{O}\left(h^6\right). \tag{127}$$

Now apply the same central sum to y'':

$$y''(x+h) + y''(x-h) = 2y'' + h^2y^{(4)} + \mathcal{O}\left(h^4\right). \tag{128}$$

Multiplying Eq. (128) by $h^2/12$ and subtracting it from Eq. (127) eliminates $y^{(4)}$:

$$y(x+h) + y(x-h) - \frac{h^2}{12}\left[y''(x+h) + y''(x-h)\right] = 2y + \frac{5h^2}{6}y'' + \mathcal{O}\left(h^6\right). \tag{129}$$

Let

$$x_i = x_0 + ih, \qquad y_i = y(x_i), \qquad f_i = f(x_i), \qquad g_i = g(x_i). \tag{130}$$

Substitute $y_i'' = f_iy_i + g_i$ into Eq. (129). After collecting the terms with y_{i+1}, y_i, and y_{i-1}, one obtains

$$\boxed{\left(1 - \frac{h^2}{12}f_{i+1}\right)y_{i+1} - 2\left(1 + \frac{5h^2}{12}f_i\right)y_i + \left(1 - \frac{h^2}{12}f_{i-1}\right)y_{i-1} = \frac{h^2}{12}\left(g_{i+1} + 10g_i + g_{i-1}\right)} \tag{131}$$

with local error $\mathcal{O}(h^6)$.

Physical significance: Numerov's method advances a linear second-order equation with high accuracy while requiring only one new evaluation of the known coefficients at the next grid point.

Because Eq. (131) is symmetric, it naturally uses equally spaced points. The step size can still be changed in factors of two: to halve h, compute an intermediate point and continue on the finer grid; to double h, skip every other point in the next stencil. Like all multi-point methods, Numerov also needs a startup procedure, because the values at two neighboring points must be known before the recurrence can begin.

7.3 Boundary-Value Problems and Shooting

Physical Motivation

An initial-value problem gives all needed data at one endpoint and asks us to march forward. Many physical problems instead specify conditions at two different places: a wavefunction must be regular at the origin and decay at infinity, a string may be fixed at both ends, and a stellar model must be regular at the center and have vanishing pressure at the surface. These are boundary-value problems.

For a second-order equation written as a first-order system,

$$\frac{dy}{dx} = F(x, y, z), \tag{132}$$

$$\frac{dz}{dx} = G(x, y, z), \tag{133}$$

an initial-value problem might specify both $y(x_1)$ and $z(x_1)$. A boundary-value problem may instead specify

$$y(x_1) = a, \qquad y(x_2) = b, \tag{134}$$

leaving the initial derivative-like variable $z(x_1)$ unknown.

The shooting method treats the missing initial datum as a parameter. Choose a trial value

$$z(x_1) = s, \tag{135}$$

integrate the initial-value problem to x_2, and define the mismatch

$$\Delta(s) = y(x_2; s) - b. \tag{136}$$

The correct initial slope satisfies

$$\boxed{\Delta(s) = 0} \tag{137}$$

Physical significance: shooting converts a boundary-value problem into a root-finding problem for the unknown initial data.

For an N-dimensional first-order system

$$\frac{dy_i}{dx} = f_i(x, y_1, \ldots, y_N), \qquad i = 1, \ldots, N, \tag{138}$$

suppose N_1 boundary conditions are imposed at x_1 and N_2 are imposed at x_2, with

$$N = N_1 + N_2. \tag{139}$$

The boundary conditions need not be simple values. They may be relations

$$B_j^{(1)}(x_1, y_1, \ldots, y_N) = 0, \qquad j = 1, \ldots, N_1, \tag{140}$$

$$B_j^{(2)}(x_2, y_1, \ldots, y_N) = 0, \qquad j = 1, \ldots, N_2. \tag{141}$$

Shooting guesses the N_2 missing degrees of freedom at x_1, integrates to x_2, and solves the N_2 mismatch equations there.

Physical significance: the number of guessed initial quantities must match the number of missing right-boundary constraints; otherwise the boundary-value problem is either underdetermined or overdetermined.

7.4 Shooting to a Fitting Point

Physical Motivation

Sometimes integration from the left endpoint all the way to the right is numerically unstable or impossible because of singular behavior. In such cases it is better to integrate from both ends toward a common point and demand that the two partial solutions meet there.

Let x_m be an interior fitting point. Integrate from x_1 to x_m using unknown parameters $\mathbf{s}_L$, and integrate from x_2 back to x_m using unknown parameters $\mathbf{s}_R$. The matching condition is

$$\boxed{\mathbf{y}_L(x_m; \mathbf{s}_L) - \mathbf{y}_R(x_m; \mathbf{s}_R) = \mathbf{0}} \tag{142}$$

Physical significance: shooting to a fitting point avoids forcing a single integration through a region where the numerical solution would become badly conditioned.

This method is especially useful when each boundary selects a locally stable branch of the solution. Each side is integrated in the direction in which it behaves well, and the root-finding problem adjusts the free parameters until the two branches join smoothly.

7.5 White-Dwarf Structure as a Boundary-Value Problem

Physical Motivation

A white dwarf is supported by degeneracy pressure. In the simplified model used here, the pressure depends only on density and not on temperature. The star is therefore a clean example of a boundary-value problem: gravity pulls inward, the pressure gradient pushes outward, the center must be regular, and the pressure must vanish at the surface.

Use the enclosed mass m as the independent variable. Hydrostatic equilibrium may be written as

$$\frac{dP}{dm} = -\frac{Gm}{4\pi r^4}, \tag{143}$$

where P is pressure, G is Newton's constant, and $r(m)$ is the radius enclosing mass m. The equation of state is approximated by a polytrope,

$$\boxed{P = K\rho^\gamma} \tag{144}$$

Physical significance: the exponent γ measures how strongly the pressure rises when the star is compressed.

For nonrelativistic degenerate matter,

$$\gamma = \frac{5}{3}, \tag{145}$$

while in the relativistic limit,

$$\gamma = \frac{4}{3}. \tag{146}$$

The lecture used an order-of-magnitude estimate to understand the mass-radius relation. Approximate the central pressure scale by P/M in Eq. (143), and estimate the density by

$$\rho \sim \frac{M}{\frac{4\pi}{3}R^3}, \tag{147}$$

where M and R are the total mass and outer radius. Then

$$\frac{K\rho^\gamma}{M} \sim \frac{GM}{4\pi R^4}. \tag{148}$$

Substituting Eq. (147) gives the scaling relation

$$K\left(\frac{M}{R^3}\right)^\gamma \sim \frac{GM^2}{R^4}. \tag{149}$$

Rearranging,

$$R^{4-3\gamma} \sim \frac{G}{K}M^{2-\gamma}. \tag{150}$$

For $\gamma = 5/3$,

$$\boxed{R \propto M^{-1/3}} \tag{151}$$

Physical significance: a more massive nonrelativistic white dwarf is smaller, because stronger gravity requires a higher density and therefore a smaller radius.

For $\gamma = 4/3$, the radius drops out of the scaling relation:

$$M^{2-\gamma} = M^{2/3} \sim \text{constant}. \tag{152}$$

Thus the model predicts a special mass scale rather than a free mass-radius curve:

$$\boxed{M \sim M_{\text{Ch}}} \tag{153}$$

Physical significance: in the relativistic degenerate limit, increasing density no longer raises pressure fast enough to balance gravity for arbitrary mass. This is the origin of the Chandrasekhar-limit behavior.

The estimate is only qualitative, because it replaces a differential equation by a global scaling argument. The actual numerical task is to solve the discretized hydrostatic equilibrium equations.

7.6 Relaxation Discretization for the White Dwarf

Physical Motivation

Shooting changes a few initial parameters and repeatedly integrates across the domain. Relaxation takes a more global view: guess the entire solution on a grid and then adjust all grid values until every discrete equation and boundary condition is satisfied at once.

Divide the star into N mass shells. Let

$$m_i = \frac{i}{N} M, \qquad i = 1, \ldots, N, \tag{154}$$

and use the shell radii r_i as the unknowns. The center condition is

$$r_0 = 0, \qquad m_0 = 0. \tag{155}$$

The pressure in shell i is

$$P_i = K \rho_i^{\gamma}, \tag{156}$$

where the shell density is mass divided by shell volume:

$$\rho_i = \frac{m_i - m_{i-1}}{\frac{4\pi}{3}\left(r_i^3 - r_{i-1}^3\right)}. \tag{157}$$

The finite-difference pressure gradient around shell interface i is

$$\frac{\Delta P_i}{\Delta m_i} = \frac{P_{i+1} - P_i}{\frac{1}{2}(m_{i+1} + m_i) - \frac{1}{2}(m_i + m_{i-1})}. \tag{158}$$

For the uniform mass grid this denominator is simply M/N.

Define the residual

$$F_i(\mathbf{r}) = \frac{\Delta P_i}{\Delta m_i}\left(\frac{4\pi r_i^4}{G m_i}\right) + 1, \qquad i = 1, \ldots, N. \tag{159}$$

The discrete hydrostatic equilibrium problem is

$$\boxed{F_i(\mathbf{r}) = 0, \qquad i = 1, \ldots, N} \tag{160}$$

Physical significance: each residual measures how far one mass shell is from balancing the pressure gradient against gravity.

At the outer boundary the pressure outside the star is zero. This is encoded by

$$P_{N+1} = 0, \qquad m_{N+1} = m_N = M. \tag{161}$$

Together with the center conditions, the N equations determine the N unknown radii $r_1, \ldots, r_N$.

Newton's method for the vector residual is

$$\sum_{j=1}^{N} \frac{\partial F_i}{\partial r_j} \Delta r_j = -F_i(\mathbf{r}), \qquad i = 1, \ldots, N. \tag{162}$$

After solving this linear system, update

$$r_i^{\text{new}} = r_i + \Delta r_i. \tag{163}$$

The derivatives $\partial F_i/\partial r_j$ should be evaluated analytically, because the residuals depend only on neighboring shell radii and the resulting Jacobian is structured.

Physical significance: relaxation solves for the whole stellar profile at once; the price is that each iteration requires a linear solve with the Jacobian matrix.

The initial guess requested in the lecture is a linear radius profile,

$$r_i^{(0)} = \frac{i}{N} R_{\text{guess}}, \tag{164}$$

with $N = 100$ for the main calculations. Because the first guess can be far from the solution, the Newton correction must be damped if it would make the radius grid nonmonotonic. Use

$$r_i^{\text{new}} = r_i + \alpha \Delta r_i, \qquad 0 < \alpha \leq 1, \tag{165}$$

and choose α small enough that

$$r_{i+1}^{\text{new}} > r_i^{\text{new}} \qquad \text{for all } i. \tag{166}$$

Physical significance: shell crossing would imply negative or infinite densities, so damping the Newton step preserves the physical ordering of the mass shells.

The lecture also emphasizes a practical lesson: a black-box nonlinear solver from a library may fail when the initial guess is too far from the physical solution. A custom relaxation algorithm can include problem-specific damping and monotonicity checks, which makes it more robust for this stellar structure problem.

7.7 Computational Tasks from the Lecture

Physical Motivation

The homework tasks are designed to connect the numerical algorithms to a real physical model. The point is not only to obtain numbers, but also to test whether the numerical method reproduces the expected scaling laws and where the simplified physics begins to fail.

The requested solver should:

1. implement a general Newton solver for N equations in N unknowns, using both a residual function $\mathbf{F}(\mathbf{x})$ and an analytic Jacobian $\partial F_i/\partial x_j$;
2. test this solver on a problem with a known analytic solution;
3. implement the white-dwarf residuals Eq. (159) and their analytic Jacobian;
4. solve the nonrelativistic case $\gamma = 5/3$ for $M = 1M_\odot$, $R_{\text{guess}} = 1R_\odot$, and $N = 100$;
5. plot and tabulate $\rho(r)$ in CGS units;
6. solve several masses and check whether $RM^{1/3}$ is approximately constant;
7. solve the nearly relativistic case $\gamma = 1.334$ for masses near $1.4M_\odot$ and inspect how the outer radius changes;
8. compare with a SciPy nonlinear solver, noting that it may require an initial radius guess closer to the final solution.

The central diagnostic for the nonrelativistic sequence is

$$\boxed{RM^{1/3} \approx \text{constant}} \tag{167}$$

Physical significance: this numerical check tests whether the full discrete hydrostatic model reproduces the scaling argument $R \propto M^{-1/3}$.

For the nearly relativistic sequence, small changes in mass can produce large changes in radius:

$$\gamma = 1.334, \qquad M/M_\odot = 1.40, 1.41, \ldots, 1.46. \tag{168}$$

Physical significance: the sensitivity near $\gamma = 4/3$ reflects the approach to the limiting-mass behavior discussed in Eq. (153).

Recommended Reading

- *Computational Physics*, Mark Newman — Ch. 8.6–8.8 (boundary-value problems and relaxation ideas).
- *Numerical Recipes*, W.H. Press et al. — Ch. 17.4 and Ch. 18.1–18.2 (stiff equations, shooting, relaxation, and boundary value problems).
- Tao Pang, *An Introduction to Computational Physics* — Ch. 3.5–3.6 (Numerov's method and boundary-value problems).
- S. Chandrasekhar, *An Introduction to the Study of Stellar Structure* — classic background on polytropes and white-dwarf mass-radius relations.

8 Lecture 08 — Multigrid Method and Introduction to Partial Differential Equations

Date: 10/05/2026

Overview

Lecture 7 introduced the relaxation method for boundary-value problems; the Gauss–Seidel iteration was identified as the workhorse. This lecture begins by explaining *why* Gauss–Seidel converges slowly on large grids and how the *multigrid* algorithm repairs the deficiency by combining solutions on grids of different spacings. The second half of the lecture opens a new chapter: *partial differential equations (PDEs)*. We classify PDEs into elliptic, parabolic, and hyperbolic types, discuss boundary and initial conditions, discretise the two-dimensional Poisson equation, and study the Jacobi, Gauss–Seidel, and successive over-relaxation (SOR) iterative solvers.

8.1 Motivation: Why Gauss–Seidel Converges Slowly

Physical Motivation

The Gauss–Seidel iteration at every grid point replaces $u_{i,j}$ by the average of its four neighbours. This operation is a local smoother: it rapidly damps oscillations whose wavelength is comparable to the grid spacing h, but it barely touches long-wavelength errors, whose smoothing rate is proportional to h^2. For an $N \times N$ grid with $h = 1/N$, reaching a fixed accuracy requires $\mathcal{O}(N^2)$ iterations, each costing $\mathcal{O}(N^2)$ work, giving a total cost of $\mathcal{O}(N^4)$ — prohibitive even for moderate N.

The key observation is that, on a coarser grid with spacing $H = 2h$, the same long-wavelength mode is a *short*-wavelength mode and is therefore damped quickly by Gauss–Seidel. Multigrid exploits this by alternating between fine and coarse grids.

8.2 The Multigrid Algorithm

Physical Motivation

We want to solve the linear problem

$$\mathcal{L}_h u_h = f_h, \tag{169}$$

where $\mathcal{L}_h$ is the discrete elliptic operator on a grid of spacing h (for example $\mathcal{L}_h = \Delta_h^2$ for the Poisson equation, $f_h = \rho_h$).

Key quantities. Let $\tilde{u}_h$ denote the current approximation. Define:

$$v_h = u_h - \tilde{u}_h \quad \text{(error)}, \tag{170}$$

$$d_h = \mathcal{L}_h \tilde{u}_h - f_h \quad \text{(defect / residual)}. \tag{171}$$

Because $\mathcal{L}_h$ is linear,

$$\mathcal{L}_h v_h = -d_h. \tag{172}$$

Physical significance: instead of solving for the full solution u_h, we need only solve for the error correction v_h, which satisfies the same type of equation driven by the current residual.

Coarse-grid correction. Pass to a grid of spacing $H = 2h$. Define the *restriction* operator $\mathcal{R}$ that maps a fine-grid function to the coarse grid:

$$d_H = \mathcal{R}\, d_h. \tag{173}$$

Solve the coarse-grid error equation approximately to obtain $\tilde{v}_H$, then *prolong* the correction back to the fine grid using the prolongation operator $\mathcal{P}$:

$$\tilde{v}_h = \mathcal{P}\, \tilde{v}_H. \tag{174}$$

Update the fine-grid approximation:

$$\boxed{\tilde{u}_h^{\text{new}} = \tilde{u}_h + \tilde{v}_h} \tag{175}$$

Physical significance: the coarse grid corrects the smooth (long-wavelength) components of the error at low cost because on the coarse grid these components are short-wavelength and are smoothed efficiently.

One multigrid cycle.

1. **Pre-smoothing**: apply i_1 Gauss–Seidel iterations to Eq. (169) to obtain $\tilde{u}_h$.
2. **Compute defect**: $d_h = \mathcal{L}_h \tilde{u}_h - f_h$.
3. **Restrict**: $d_H = \mathcal{R}\, d_h$.
4. **Solve coarse**: obtain $\tilde{v}_H \approx -\mathcal{L}_H^{-1} d_H$ (recursively or by direct solve at the coarsest level, marked E in the cycle diagrams).
5. **Prolong**: $\tilde{v}_h = \mathcal{P}\, \tilde{v}_H$.
6. **Update**: $\tilde{u}_h^{\text{new}} = \tilde{u}_h + \tilde{v}_h$.
7. **Post-smoothing**: apply i_1 Gauss–Seidel iterations.

The full recursion over L grid levels is a *V-cycle* (one visit to each coarser level) or a *W-cycle* (two visits). For an $N \times N$ grid:

$$\boxed{\text{Total cost per multigrid cycle} = \mathcal{O}\big(N^2\big)} \tag{176}$$

Physical significance: because each cycle reduces all error components by a fixed factor independent of N, a fixed number of cycles suffices for any prescribed accuracy, making the overall algorithm $\mathcal{O}(N^2)$ — optimal up to the unavoidable cost of writing the answer.

8.3 Classification of Partial Differential Equations

Physical Motivation

Many fundamental laws of physics — electrostatics, heat conduction, fluid mechanics, quantum mechanics — are expressed as PDEs. The numerical strategy (explicit vs implicit time-stepping, type of boundary condition, stability criterion) depends strongly on which class a PDE belongs to.

A second-order PDE in two variables is classified according to the sign of its discriminant. The three canonical types and their physical representatives are:

- **Elliptic** (both eigenvalues same sign): stationary problems. Prototype: Poisson equation

$$\boxed{\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = \rho(x, y)} \tag{177}$$

Physical significance: describes electrostatic potential with charge density ρ, steady-state temperature, etc. The solution at each point is influenced by the entire domain — "action at a distance".

- **Parabolic** (one zero eigenvalue): diffusion/heat problems with a first-order time derivative. Prototype:

$$\boxed{\frac{\partial u}{\partial t} = \frac{\partial}{\partial x}\left(D\frac{\partial u}{\partial x}\right)} \tag{178}$$

Physical significance: information propagates at infinite speed; the solution smooths out irregularities exponentially fast.

- **Hyperbolic** (eigenvalues of opposite sign): wave problems with a second-order time derivative. Prototype:

$$\boxed{\frac{\partial^2 u}{\partial t^2} = v^2\frac{\partial^2 u}{\partial x^2}} \tag{179}$$

Physical significance: information propagates at finite speed v; wave fronts and shocks can form.

8.4 Boundary Conditions

Physical Motivation

For an elliptic PDE defined in a domain, the solution is uniquely determined once appropriate boundary conditions are supplied. Two standard types are:

- **Dirichlet**: the function value is specified on the boundary, $u\big|_{\partial\Omega} = g$.
- **Neumann**: the normal derivative is specified, $\partial u/\partial n\big|_{\partial\Omega} = g$.

For *initial-value* problems (parabolic and hyperbolic) the function and, where relevant, its time derivative are given on the initial surface and boundary conditions are applied on the spatial boundary.

8.5 Discretising the 2D Poisson Equation

Physical Motivation

We want to solve $\partial_x^2 u + \partial_y^2 u = \rho(x, y)$ numerically on a square domain $[0, 1]^2$. The approach is to replace the partial derivatives by finite differences on a uniform grid.

Grid. Divide each axis into N equal intervals of width $h = 1/N$. Let

$$u_{i,j} = u(x_i, y_j), \qquad \rho_{i,j} = \rho(x_i, y_j), \qquad x_i = ih,\ y_j = jh. \tag{180}$$

Applying the symmetric second-derivative formula in both directions gives the *five-point stencil*:

$$\boxed{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j} = h^2\rho_{i,j}} \tag{181}$$

with local truncation error $\mathcal{O}(h^2)$.

Physical significance: each interior grid point produces one equation; with $N \times N$ unknowns and Dirichlet boundary conditions on all sides, the system is square with N^2 equations and N^2 unknowns. In matrix form it is a sparse, banded, negative-definite system. Even for $N = 100$ this is a $10\,000 \times 10\,000$ matrix — direct Gaussian elimination costs $\mathcal{O}(N^6)$ which is too expensive; iterative solvers are essential.

8.6 Iterative Solvers for the Poisson System

Physical Motivation

The five-point system Eq. (181) can be solved by rearranging for $u_{i,j}$:

$$u_{i,j} = \tfrac{1}{4}(u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1}) - \tfrac{h^2}{4}\rho_{i,j}. \tag{182}$$

This is a *local average* — the solution at each point is the mean of its four neighbours minus a source term. Iterating this formula gives three increasingly powerful methods.

Jacobi iteration. All grid values are updated simultaneously using only old values:

$$u_{i,j}^{n+1} = \tfrac{1}{4}\big(u_{i+1,j}^{n} + u_{i-1,j}^{n} + u_{i,j+1}^{n} + u_{i,j-1}^{n}\big) - \tfrac{h^2}{4}\rho_{i,j}. \tag{183}$$

Convergence requires $\frac{1}{2}pN^2$ iterations for precision 10^{-p}.

Gauss–Seidel iteration with red-black ordering. Update $u_{i,j}$ using the most recent values of its neighbours. Using a *red-black* (checkerboard) ordering — first all $(i+j)$-even nodes ("red"), then all $(i+j)$-odd nodes ("black") — the red nodes depend only on black neighbours and vice versa, so each colour can be updated simultaneously:

$$u_{i,j}^{n+1} = \tfrac{1}{4}\big(u_{i+1,j}^{n+1} + u_{i-1,j}^{n+1} + u_{i,j+1}^{n+1} + u_{i,j-1}^{n+1}\big) - \tfrac{h^2}{4}\rho_{i,j} \tag{184}$$

(where newly updated values are used as soon as they are available). Gauss–Seidel converges roughly twice as fast as Jacobi.

Successive Over-Relaxation (SOR). Compute the Gauss–Seidel update $u_{i,j}^{*}$ and then overshoot:

$$u_{i,j}^{n+1} = w\,u_{i,j}^{*} + (1-w)\,u_{i,j}^{n}, \qquad 1 < w < 2. \tag{185}$$

$$\boxed{w_{\text{opt}} \approx \frac{2}{1+\sin(\pi/N)}} \tag{186}$$

Physical significance: with optimal w, SOR needs only $\mathcal{O}(N)$ iterations instead of $\mathcal{O}(N^2)$, reducing the total cost for the Poisson equation to $\mathcal{O}(N^3)$. For $w = 1$ one recovers plain Gauss–Seidel; $w = 2$ is unstable. Under-relaxation ($0 < w < 1$) is used when the plain Gauss–Seidel iterations diverge (e.g. for nonlinear problems).

Recommended Reading

- *Computational Physics*, Mark Newman — Ch. 9.1–9.3 (relaxation, Gauss–Seidel, over-relaxation for PDEs).
- *Numerical Recipes*, W.H. Press et al. — Ch. 19.1 (relaxation methods for PDEs) and Ch. 20.6 (multigrid methods).
- K.W. Morton & D.F. Mayers, *Numerical Solution of Partial Differential Equations* — Ch. 2–3 (classification and elliptic equations).

9 Lecture 09 — Time-Dependent PDEs: Diffusion, Schrödinger Equation, and Advection

Date: 17/05/2026

Overview

Having classified PDEs and studied the elliptic (stationary) Poisson equation in Lecture 8, we now turn to time-dependent problems. The diffusion equation is parabolic and requires a time integration scheme; we derive the explicit FTCS scheme, prove its stability condition by von Neumann analysis, introduce the unconditionally stable fully-implicit (BTCS) scheme, and derive the *Crank–Nicholson* method that is second-order in both space and time. We then extend to two spatial dimensions via the Alternating-Direction Implicit (ADI) method. As a quantum-mechanical application, we apply the same ideas to the time-dependent Schrödinger equation and derive the norm-conserving *Cayley form*. The lecture closes with the hyperbolic advection equation and the Lax scheme together with the Courant–Friedrichs–Lewy (CFL) stability condition.

9.1 The 1D Diffusion Equation and FTCS

Physical Motivation

Heat conduction in a rod, diffusion of a chemical species, or random walks on a lattice all lead to the diffusion equation. We want to compute how an initial concentration profile $u(x, 0)$ spreads in time.

The 1D diffusion equation with constant diffusivity D is

$$\frac{\partial u}{\partial t} = D\frac{\partial^2 u}{\partial x^2}. \tag{187}$$

Discretise with time step Δt and space step Δx. Use the *Forward Time, Centered Space* (FTCS) scheme: the time derivative is a forward difference at level n, and the spatial second derivative is the symmetric difference at the same level:

$$\boxed{\frac{u_j^{n+1} - u_j^n}{\Delta t} = D\frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{(\Delta x)^2}} \tag{188}$$

Rearranging gives an explicit update:

$$u_j^{n+1} = u_j^n + \alpha\big(u_{j+1}^n - 2u_j^n + u_{j-1}^n\big), \qquad \alpha \equiv \frac{D\Delta t}{(\Delta x)^2}. \tag{189}$$

Physical significance: α is the ratio of the diffusion time across one spatial cell to the time step. When α is too large, the scheme over-corrects local gradients and becomes unstable.

9.2 Von Neumann Stability Analysis

Physical Motivation

We need a systematic criterion for choosing Δt given Δx. Von Neumann analysis decomposes the numerical error into Fourier modes and asks whether each mode grows or decays.

Substitute the ansatz $u_j^n = \xi^n e^{ikj\Delta x}$ into Eq. (188). Here $\xi(k)$ is the *amplification factor* for wavenumber k. For the FTCS scheme one obtains:

$$\begin{aligned}\xi^n e^{ikj\Delta x}(\xi - 1) &= \alpha\, \xi^n e^{ikj\Delta x}\left(e^{ik\Delta x} - 2 + e^{-ik\Delta x}\right) \\ &= \alpha\, \xi^n e^{ikj\Delta x}\left(e^{i(k\Delta x)/2} - e^{-i(k\Delta x)/2}\right)^2,\end{aligned} \tag{190}$$

giving

$$\boxed{\xi(k) = 1 - 4\alpha \sin^2\!\left(\frac{k\Delta x}{2}\right)} \tag{191}$$

The scheme is stable when $|\xi(k)| \le 1$ for all k. The most dangerous mode is $k = \pi/\Delta x$ (shortest wavelength), where $\xi = 1 - 4\alpha$. Requiring $\xi \ge -1$ gives:

$$\boxed{\alpha \le \tfrac{1}{2}, \qquad \text{i.e.} \quad \Delta t \le \frac{(\Delta x)^2}{2D}} \tag{192}$$

Physical significance: the allowed time step shrinks as the square of the grid spacing. Halving Δx forces a fourfold reduction in Δt and an eightfold increase in computational work — explicit methods are expensive for the diffusion equation on fine grids. The characteristic diffusion time across a cell is $\tau = (\Delta x)^2/D$; the stability condition says we must take many time steps per diffusion time.

9.3 Fully Implicit (BTCS) Scheme

Physical Motivation

To remove the stability restriction one evaluates the spatial derivatives at the *new* time level $n+1$ rather than at the current level n.

The *Backward Time, Centered Space* (BTCS) scheme is

$$\boxed{\frac{u_j^{n+1} - u_j^n}{\Delta t} = D\frac{u_{j+1}^{n+1} - 2u_j^{n+1} + u_{j-1}^{n+1}}{(\Delta x)^2}} \tag{193}$$

Rearranging, with $\alpha = D\Delta t/(\Delta x)^2$:

$$-\alpha u_{j-1}^{n+1} + (1 + 2\alpha)u_j^{n+1} - \alpha u_{j+1}^{n+1} = u_j^n, \qquad j = 1, 2, \ldots, N-1. \tag{194}$$

This is a tridiagonal system for the unknowns $\{u_j^{n+1}\}$, solvable in $\mathcal{O}(N)$ operations.

Von Neumann analysis gives the amplification factor

$$\xi = \frac{1}{1 + 4\alpha \sin^2(k\Delta x/2)}, \tag{195}$$

so $|\xi| < 1$ for all k and all $\alpha > 0$:

$$\boxed{\text{BTCS is unconditionally stable.}} \tag{196}$$

Physical significance: one may take arbitrarily large time steps without numerical instability. However, the accuracy in time is only first order, so large Δt produces inaccurate results even if the scheme is stable — stability is necessary but not sufficient for a good numerical solution.

9.4 Crank–Nicholson Scheme

Physical Motivation

The FTCS scheme is first-order in time and conditionally stable. The BTCS scheme is first-order in time and unconditionally stable. Averaging them yields a scheme that is second-order in time *and* unconditionally stable.

The *Crank–Nicholson* scheme evaluates the spatial diffusion term as the mean of the nth and $(n+1)$th levels:

$$\boxed{\frac{u_j^{n+1}-u_j^n}{\Delta t}=\frac{D}{2(\Delta x)^2}\left[\left(u_{j+1}^{n+1}-2u_j^{n+1}+u_{j-1}^{n+1}\right)+\left(u_{j+1}^{n}-2u_j^{n}+u_{j-1}^{n}\right)\right]} \tag{197}$$

This is again a tridiagonal system. Von Neumann analysis gives

$$\boxed{\xi=\frac{1-2\alpha\sin^2(k\Delta x/2)}{1+2\alpha\sin^2(k\Delta x/2)}} \tag{198}$$

Since $|\xi|<1$ for all $\alpha>0$ and all k, the scheme is unconditionally stable. Because the stencil is symmetric about time $n+\frac{1}{2}$, the local truncation error is $\mathcal{O}((\Delta t)^2)$ — the scheme is second-order accurate in time.

Physical significance: Crank–Nicholson is the gold standard for the diffusion equation: unconditionally stable, second-order in time, and only modestly more expensive than BTCS.

9.5 2D Diffusion and the ADI Method

Physical Motivation

Extending Crank–Nicholson to two spatial dimensions,

$$\partial_t u = D(\partial_x^2 u+\partial_y^2 u), \tag{199}$$

leads to a large system that is no longer tridiagonal; its direct solution is expensive. The *Alternating Direction Implicit* (ADI) method splits each time step into two half-steps, treating one direction implicitly at each half-step.

Define $\alpha = D\Delta t/\Delta^2$ with $\Delta x=\Delta y=\Delta$. Introduce the finite-difference operators

$$\delta_x^2 u_{i,j}\equiv u_{i-1,j}-2u_{i,j}+u_{i+1,j},\qquad \delta_y^2 u_{i,j}\equiv u_{i,j-1}-2u_{i,j}+u_{i,j+1}. \tag{200}$$

The two ADI half-steps are

$$u_{i,j}^{n+1/2}=u_{i,j}^n+\frac{\alpha}{2}\left(\delta_x^2 u_{i,j}^{n+1/2}+\delta_y^2 u_{i,j}^n\right), \tag{201}$$

$$u_{i,j}^{n+1}=u_{i,j}^{n+1/2}+\frac{\alpha}{2}\left(\delta_x^2 u_{i,j}^{n+1/2}+\delta_y^2 u_{i,j}^{n+1}\right). \tag{202}$$

In each half-step the implicit direction reduces to a set of independent tridiagonal solves along lines parallel to the implicit axis, keeping the cost $\mathcal{O}(N^2)$ per full step.

$$\boxed{\text{ADI: unconditionally stable; } \mathcal{O}\big(N^2\big) \text{ per step}} \tag{203}$$

Physical significance: ADI is an instance of *operator splitting*: a complicated multi-dimensional problem is broken into a sequence of simpler one-dimensional problems. Operator splitting underlies many algorithms in fluid mechanics and plasma physics.

9.6 Time-Dependent Schrödinger Equation

Physical Motivation

The time-dependent Schrödinger equation governs the quantum-mechanical evolution of a wave packet:

$$i\partial_t \psi = H\psi, \qquad H = -\partial_x^2 + V(x), \tag{204}$$

in units $\hbar = 2m = 1$. Any good numerical scheme must conserve the norm $\int |\psi|^2\, dx = 1$ at all times.

FTCS applied to Schrödinger. The FTCS update is

$$\psi_j^{n+1} = (1 - iH\Delta t)\psi_j^n, \tag{205}$$

with von Neumann amplification factor $|\xi| > 1$ for every k — the FTCS scheme is *always unstable* for the Schrödinger equation and cannot conserve the norm.

Cayley form (Crank–Nicholson for Schrödinger). Apply Crank–Nicholson to Eq. (204):

$$\left(1 + i\frac{H\Delta t}{2}\right)\psi^{n+1} = \left(1 - i\frac{H\Delta t}{2}\right)\psi^n. \tag{206}$$

This is known as the *Cayley form*:

$$\boxed{\psi^{n+1} = \frac{1 - i(H\Delta t/2)}{1 + i(H\Delta t/2)}\,\psi^n} \tag{207}$$

The factor in front of ψ^n is a *unitary* operator because $(1 - iz)/(1 + iz)$ has modulus one for any real z. Hence:

$$\boxed{\left\|\psi^{n+1}\right\| = \|\psi^n\| \quad \text{(norm conserved exactly)}} \tag{208}$$

Physical significance: the Cayley/Crank–Nicholson scheme exactly preserves unitarity at the discrete level. In practice it reduces to a tridiagonal solve at each time step — the same structure as BTCS for the diffusion equation, but now the matrix entries are complex.

9.7 Conservation Laws and the Advection Equation

Physical Motivation

A conservation law states that the rate of change of a density u equals minus the divergence of a flux $F(u)$:

$$\partial_t u = -\partial_x F(u). \tag{209}$$

The diffusion equation is a conservation law with $F = -D\partial_x u$. The simplest hyperbolic conservation law is the *advection equation*, which describes a wave profile moving rigidly at speed v:

$$\partial_t u = -v\,\partial_x u, \qquad u(x,t) = f(x - vt). \tag{210}$$

FTCS for advection — always unstable. Applying FTCS to Eq. (210):

$$u_j^{n+1} = u_j^n - \frac{v\Delta t}{2\Delta x}\left(u_{j+1}^n - u_{j-1}^n\right). \tag{211}$$

Von Neumann analysis gives $\xi = 1 - i\frac{v\Delta t}{\Delta x}\sin(k\Delta x)$, so $|\xi|^2 = 1 + \left(\frac{v\Delta t}{\Delta x}\right)^2 \sin^2(k\Delta x) > 1$ for all non-zero v: the FTCS scheme is unconditionally unstable for advection.

Lax scheme. Replace u_j^n in the time derivative by the spatial average of its neighbours:

$$u_j^{n+1} = \tfrac{1}{2}\big(u_{j+1}^n + u_{j-1}^n\big) - \frac{v\Delta t}{2\Delta x}\big(u_{j+1}^n - u_{j-1}^n\big)\,. \tag{212}$$

Von Neumann analysis yields

$$|\xi|^2 = \cos^2(k\Delta x) + \left(\frac{v\Delta t}{\Delta x}\right)^2 \sin^2(k\Delta x) \le 1 \tag{213}$$

provided the *Courant–Friedrichs–Lewy (CFL) condition* holds:

$$\boxed{\frac{|v|\,\Delta t}{\Delta x} \le 1} \tag{214}$$

Physical significance: the CFL condition has a clean physical interpretation — the numerical domain of dependence of a grid point (the stencil width in space, Δx) must include the physical domain of dependence (the distance $|v|\,\Delta t$ that information travels in one time step). If Δt is too large, the stencil misses the physically relevant upstream information, causing the scheme to fail.

When the CFL condition is violated ($|v|\,\Delta t/\Delta x > 1$), information from grid points $j-1$ and $j+1$ does not reach grid point j fast enough: the numerical scheme receives data from outside the physical domain of influence, and amplification grows.

Summary of Stability Conditions

Equation	Scheme	Stability	Order
Diffusion	FTCS	$\alpha \le \frac{1}{2}$	$O(\Delta t,\, \Delta x^2)$
Diffusion	BTCS	unconditional	$O(\Delta t,\, \Delta x^2)$
Diffusion	Crank–Nicholson	unconditional	$O(\Delta t^2,\, \Delta x^2)$
Schrödinger	Cayley form	unconditional, unitary	$O(\Delta t^2,\, \Delta x^2)$
Advection	FTCS	never stable	$O(\Delta t,\, \Delta x^2)$
Advection	Lax	CFL: $\lvert v\rvert\,\Delta t/\Delta x \le 1$	$O(\Delta t,\, \Delta x)$

Recommended Reading

- *Computational Physics*, Mark Newman — Ch. 9.3–9.4 (diffusion, stability, implicit methods).
- *Numerical Recipes*, W.H. Press et al. — Ch. 19.2 (FTCS and Crank–Nicholson for diffusion) and Ch. 19.1 (advection, CFL condition).
- K.W. Morton & D.F. Mayers, *Numerical Solution of Partial Differential Equations* — Ch. 4 (parabolic equations) and Ch. 6 (hyperbolic equations).